

```
%pip install pandas numpy seaborn matplotlib scipy pydeseq2 gprofile
!rm -drf Data
!mkdir Data
!wget https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
!wget https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
!wget https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
!ls Data/
!cat Data/metaData.csv
```

```
Downloading zarr-3.2.1-py3-none-any.whl (319 kB)
319.6/319.6 kB 9.6 MB/s
Downloading legacy_api_wrap-1.5-py3-none-any.whl (10 kB)
Downloading session_info-1.0.1-py3-none-any.whl (9.1 kB)
Downloading donfig-0.8.1.post1-py3-none-any.whl (21 kB)
Downloading numcodecs-0.16.5-cp312-cp312-manylinux2014_x86_64.many
9.2/9.2 MB 63.7 MB/s et
Downloading session_info2-0.4.1-py3-none-any.whl (17 kB)
Downloading stdlib_list-0.12.0-py3-none-any.whl (87 kB)
87.6/87.6 kB 5.0 MB/s
```

```
Installing collected packages: stdlib_list, session-info2, numcodecs
Successfully installed adjustText-1.3.0 anndata-0.12.14 array-api-0
--2026-05-17 16:26:16-- https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com):
HTTP request sent, awaiting response... 200 OK
Length: 633 [text/plain]
Saving to: 'Data/metaData.csv'
```

```
Data/metaData.csv 100%[=====] 633 --.-KB/s
```

```
2026-05-17 16:26:16 (30.1 MB/s) - 'Data/metaData.csv' saved [633/633]
```

```
--2026-05-17 16:26:16-- https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com):
HTTP request sent, awaiting response... 200 OK
Length: 2649713 (2.5M) [text/plain]
Saving to: 'Data/rsem.merged.gene_counts_2021.tsv'
```

```
Data/rsem.merged.ge 100%[=====] 2.53M --.-KB/s
```

```
2026-05-17 16:26:16 (57.6 MB/s) - 'Data/rsem.merged.gene_counts_2021.tsv' saved [2649713/2649713]
```

```
--2026-05-17 16:26:16-- https://raw.githubusercontent.com/simoneksasser/SP1-01-Ex/f0481
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com):
HTTP request sent, awaiting response... 200 OK
Length: 302480 (295K) [text/plain]
Saving to: 'Data/gene_group_mapping.csv'
```

```
Data/gene_group_map 100%[=====] 295.39K --.-KB/s
```

```
2026-05-17 16:26:17 (10.9 MB/s) - 'Data/gene_group_mapping.csv' saved [302480/302480]
```

```
gene_group_mapping.csv metaData.csv rsem.merged.gene_counts_2021.
BioSample,SRR,condition,treatment,Name
SAMN25076933,SRR17657871,naive,none,Naive_R1
SAMN25076934,SRR17657872,naive,none,Naive_R2
SAMN25076935,SRR17657873,naive,none,Naive_R3
SAMN25076936,SRR17657874,naive,EZH2i,Naive_EZH2i_R1
SAMN25076938,SRR17657875,naive,EZH2i,Naive_EZH2i_R2
SAMN25076939,SRR17657876,naive,EZH2i,Naive_EZH2i_R3
SAMN25076940,SRR17657877,primed,none,Primed_R1
SAMN25076941,SRR17657878,primed,none,Primed_R2
SAMN25076942,SRR17657879,primed,none,Primed_R3
SAMN25076943,SRR17657880,primed,EZH2i,Primed_EZH2i_R1
SAMN25076944,SRR17657881,primed,EZH2i,Primed_EZH2i_R2
SAMN25076945,SRR17657882,primed,EZH2i,Primed_EZH2i_R3
```

```
import pandas as pd
import numpy as np

#####
##### TASK 1 #####
# read in data #
#####

file_path = "Data/rsem.merged.gene_counts_2021.tsv"
# Read in count file "Data/rsem.merged.gene_counts_2021.tsv", it is
count_table = pd.read_csv(file_path, sep='\t', index_col=0)
# Print the first 5 rows of the count table
print(count_table.head())
```

	transcript_id(s) Naive_E				
gene_id					
A1BG	NM_130786				
A1BG-AS1	NR_015380				
A1CF	NM_001198818,NM_001198819,NM_001198820,NM_0145...				
A2M	NM_000014				
A2M-AS1	NR_026971				

	Naive_EZH2i_R2	Naive_EZH2i_R3	Naive_R1	Naive_R2	Naive_
gene_id					
A1BG	4.0	6.0	2.0	0.0	4
A1BG-AS1	4.0	5.0	2.0	3.0	2
A1CF	0.0	0.0	0.0	0.0	0
A2M	1.0	1.0	0.0	0.0	0
A2M-AS1	0.0	0.0	0.0	0.0	0

	Primed_EZH2i_R1	Primed_EZH2i_R2	Primed_EZH2i_R3	Primed_
gene_id				
A1BG	137.0	175.0	143.0	104
A1BG-AS1	36.0	28.0	28.0	27
A1CF	2.0	0.0	0.0	3
A2M	143.0	114.0	112.0	136
A2M-AS1	4.0	6.0	8.0	8

	Primed_R2	Primed_R3
gene_id		
A1BG	120.0	139.0
A1BG-AS1	26.0	26.0
A1CF	0.0	0.0
A2M	123.0	155.0
A2M-AS1	11.0	9.0

```
# we are going to remove 'transcript_id(s)' from the count table, a
count_table = count_table.drop('transcript_id(s)', axis=1)
# Print the head again
print(count_table.head())
```

	Naive_EZH2i_R1	Naive_EZH2i_R2	Naive_EZH2i_R3	Naive_R1
gene_id				
A1BG	3.0	4.0	6.0	2.0
A1BG-AS1	7.0	4.0	5.0	2.0
A1CF	0.0	0.0	0.0	0.0
A2M	1.0	1.0	1.0	0.0
A2M-AS1	0.0	0.0	0.0	0.0

	Naive_R3	Primed_EZH2i_R1	Primed_EZH2i_R2	Primed_EZH2i_R
gene_id				
A1BG	4.0	137.0	175.0	143.

A1BG-AS1	2.0	36.0	28.0	28.
A1CF	0.0	2.0	0.0	0.
A2M	0.0	143.0	114.0	112.
A2M-AS1	0.0	4.0	6.0	8.

	Primed_R1	Primed_R2	Primed_R3
gene_id			
A1BG	104.0	120.0	139.0
A1BG-AS1	27.0	26.0	26.0
A1CF	3.0	0.0	0.0
A2M	136.0	123.0	155.0
A2M-AS1	8.0	11.0	9.0

#Check the numbers in the count table – it looks like they are all

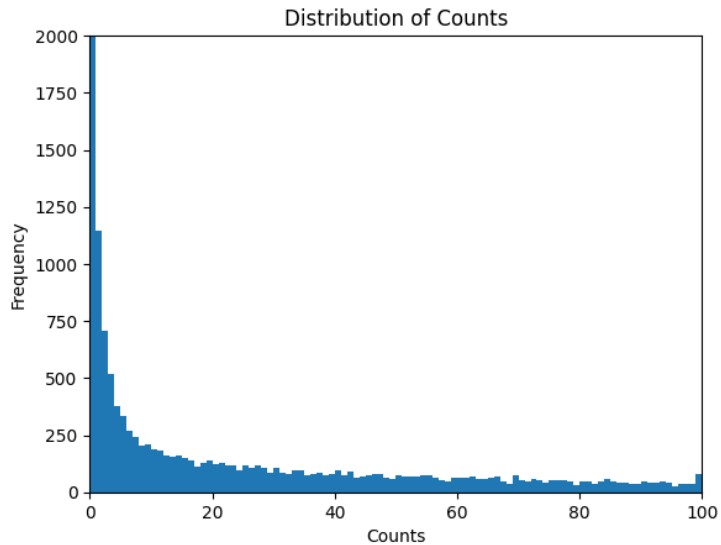
```
#Let's take the first column (Naive_EZH2i_R1)- What is the min, max
print("Min:", count_table.iloc[:, 0].min())
print("Max:", count_table.iloc[:, 0].max())
print("Mean:", count_table.iloc[:, 0].mean())
print("Median:", np.median(count_table.iloc[:, 0]))
print("Variance:", np.var(count_table.iloc[:, 0]))
print("Number of zeros:", np.sum(count_table.iloc[:, 0] == 0))
```

```
# Are there any zeros? How many genes in total and how many genes h
print("Total number of genes:", count_table.shape[0])
print("Number of genes with zero counts in all samples:", np.sum(np
```

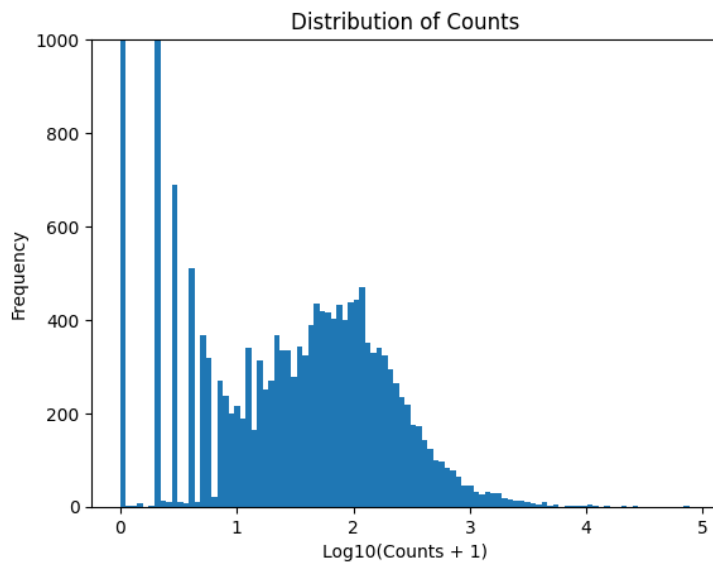
```
Min: 0.0
Max: 76778.0
Mean: 84.87063229461758
Median: 3.0
Variance: 379818.61545530043
Number of zeros: 10968
Total number of genes: 26475
Number of genes with zero counts in all samples: 6044
```

```
# What is the distribution of counts? Can you make a graph to inspe
import matplotlib.pyplot as plt

# Plot a histogram of the counts in the first column, limit the x a
# What do you observe? What does the high peak at zero and long tai
plt.hist(count_table.iloc[:, 0], bins=100, range=(0, 100))
plt.xlabel('Counts')
plt.ylabel('Frequency')
plt.title('Distribution of Counts')
plt.xlim(0, 100)
plt.ylim(0, 2000)
plt.show()
```



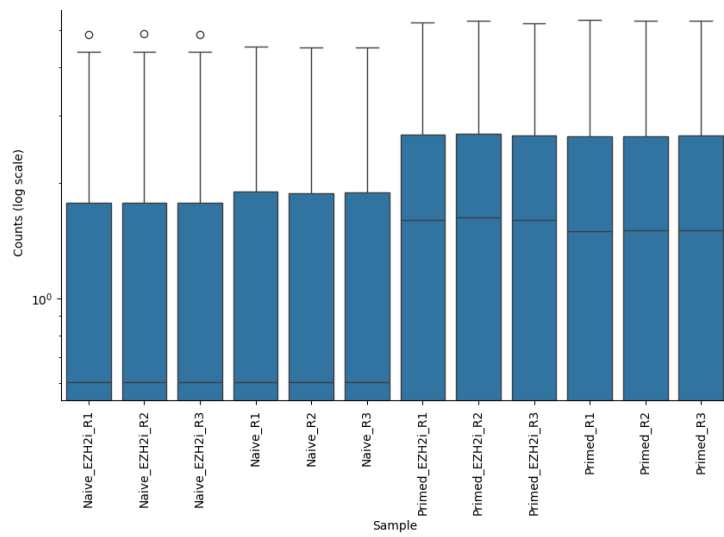
```
# Let's try to log transform the counts with log10 (add + 1 to the
plt.hist(np.log10(count_table.iloc[:, 0]+1), bins=100)
plt.xlabel('Log10(Counts + 1)')
plt.ylabel('Frequency')
plt.title('Distribution of Counts')
plt.ylim(0, 1000)
plt.show()
```



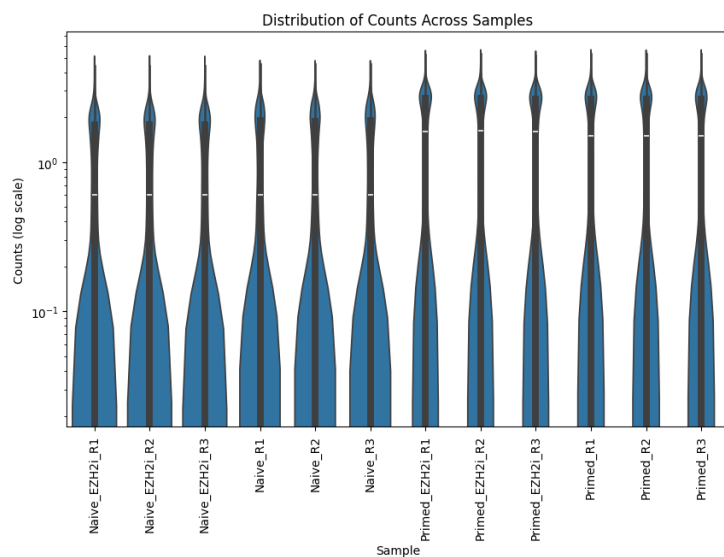
```
# For comparing gene expression, we need to understand the distribu
print(count_table.loc['NANOG', :])
print(count_table.loc['GAPDH', :])
```

```
Naive_EZH2i_R1      103.0
Naive_EZH2i_R2       82.0
Naive_EZH2i_R3       79.0
Naive_R1            778.0
Naive_R2            770.0
Naive_R3            804.0
Primed_EZH2i_R1    1541.0
Primed_EZH2i_R2    1418.0
Primed_EZH2i_R3    1411.0
Primed_R1          1214.0
Primed_R2          1124.0
Primed_R3          1183.0
Name: NANOG, dtype: float64
Naive_EZH2i_R1      8707.0
Naive_EZH2i_R2     9111.0
Naive_EZH2i_R3     8525.0
Naive_R1           10536.0
Naive_R2           9876.0
Naive_R3           10762.0
Primed_EZH2i_R1    111974.0
Primed_EZH2i_R2   125424.0
Primed_EZH2i_R3   102649.0
Primed_R1          111263.0
Primed_R2          111576.0
Primed_R3          125533.0
Name: GAPDH, dtype: float64
```

```
# Now something looks strange – the counts for "Primed" appear to b
import seaborn as sns
# Convert the count table to long format for seaborn box plot and l
count_table_long = count_table.melt(var_name='Sample', value_name='
count_table_long['Count'] = np.log10(count_table_long['Count'] + 1)
# Plot a box plot of counts for each sample
plt.figure(figsize=(10, 6))
sns.boxplot(x='Sample', y='Count', data=count_table_long)
plt.yscale('log')
plt.xlabel('Sample')
plt.ylabel('Counts (log scale)')
plt.title('Distribution of Counts Across Samples')
plt.xticks(rotation=90)
plt.show()
```



```
# Since we learned that boxplots could hide the distribution of the
# Plot a violin plot of counts for each sample
plt.figure(figsize=(10, 6))
sns.violinplot(x='Sample', y='Count', data=count_table_long)
plt.yscale('log')
plt.xlabel('Sample')
plt.ylabel('Counts (log scale)')
plt.title('Distribution of Counts Across Samples')
plt.xticks(rotation=90)
plt.show()
```

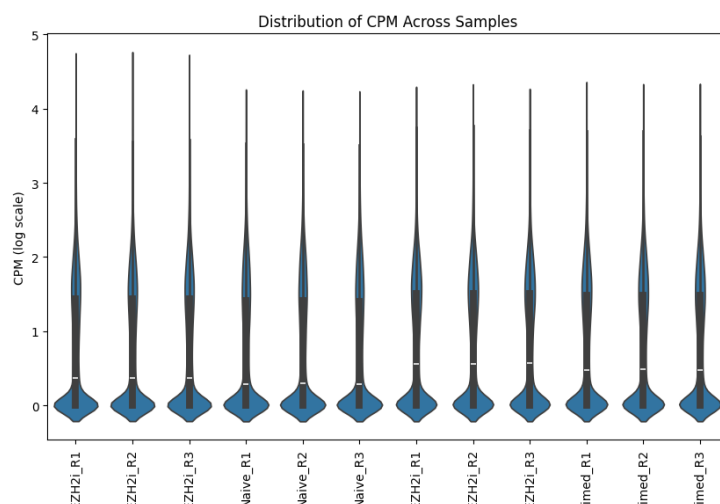


```
# Clearly there is a big difference between the "Naive" and "Primed"
# A trivial source of this difference is that libraries are sequenced
# before sequencing
# In the current dataset, the BGI generated almost exactly the same
# The "Naive" cells are grown on mouse feeder cells, so in the RNA
# we have a big contribution of mouse RNA, this will be part of the
# and as a result we appear to have less total reads.
```

```
# In order to compare gene expression between the "Naive" and "Primed"
# Calculate CPM: Normalize each row count by the total read count of
cpm_table = count_table.div(count_table.sum(axis=0), axis=1) * 1e6
# now check NANOG and GAPDH again
print(cpm_table.loc['NANOG', :])
print(cpm_table.loc['GAPDH', :])
```

```
Naive_EZH2i_R1      45.839917
Naive_EZH2i_R2      36.234815
Naive_EZH2i_R3      35.005953
Naive_R1            250.617446
Naive_R2            253.871128
Naive_R3            256.065715
Primed_EZH2i_R1     103.559824
Primed_EZH2i_R2      90.107487
Primed_EZH2i_R3     98.619887
Primed_R1           81.683712
Primed_R2           75.276751
Primed_R3           77.185017
Name: NANOG, dtype: float64
Naive_EZH2i_R1      3875.030614
Naive_EZH2i_R2      4026.041502
Naive_EZH2i_R3      3777.541155
Naive_R1            3393.965822
Naive_R2            3256.144498
Naive_R3            3427.586102
Primed_EZH2i_R1     7524.988819
Primed_EZH2i_R2     7970.128003
Primed_EZH2i_R3     7174.509404
Primed_R1           7486.305485
Primed_R2           7472.490013
Primed_R3           8190.419876
Name: GAPDH, dtype: float64
```

```
# Convert to long format for plotting and log transform
cpm_table_long = cpm_table.melt(var_name='Sample', value_name='CPM')
cpm_table_long['CPM'] = np.log10(cpm_table_long['CPM'] + 1)
# Plot a violin plot of CPM for each sample
plt.figure(figsize=(10, 6))
sns.violinplot(x='Sample', y='CPM', data=cpm_table_long)
plt.xlabel('Sample')
plt.ylabel('CPM (log scale)')
plt.title('Distribution of CPM Across Samples')
plt.xticks(rotation=90)
plt.show()
```




```

10 SAMN25076944 SRR17657881 primed EZH2i Primed_EZH2i_R2
11 SAMN25076945 SRR17657882 primed EZH2i Primed_EZH2i_R3

```

```

#####
##### TASK 2 #####
# Preparation for DESeq #
#####

```

```

# prepare metaData
# shape meta for DESeq analysis (name as index, treatment and condi
meta = experiment_design.set_index("Name")
#meta = meta[["treatment","condition"]]
# shape meta for DESeq analysis (name as index, treatment and condi
# add a column "design" that is a combination of treatment and cond
meta["design"] = meta["condition"] + "_" + meta["treatment"]
print(meta)

```

Name	BioSample	SRR	condition	treatment	
Naive_R1	SAMN25076933	SRR17657871	naive	none	na
Naive_R2	SAMN25076934	SRR17657872	naive	none	na
Naive_R3	SAMN25076935	SRR17657873	naive	none	na
Naive_EZH2i_R1	SAMN25076936	SRR17657874	naive	EZH2i	nai
Naive_EZH2i_R2	SAMN25076938	SRR17657875	naive	EZH2i	nai
Naive_EZH2i_R3	SAMN25076939	SRR17657876	naive	EZH2i	nai
Primed_R1	SAMN25076940	SRR17657877	primed	none	pri
Primed_R2	SAMN25076941	SRR17657878	primed	none	pri
Primed_R3	SAMN25076942	SRR17657879	primed	none	pri
Primed_EZH2i_R1	SAMN25076943	SRR17657880	primed	EZH2i	prim
Primed_EZH2i_R2	SAMN25076944	SRR17657881	primed	EZH2i	prim
Primed_EZH2i_R3	SAMN25076945	SRR17657882	primed	EZH2i	prim

```

# prepare count table
# filter for lowly expressed genes
genes_to_keep = count_table.columns[count_table.sum(axis=0) >= 10]
counts_df = count_table[genes_to_keep]

# convert dataframe to integer
counts_df = counts_df.astype(int)

# transpose dataframe for Deseq analysis
counts_df = counts_df.T

# tricky part: make sure that the order of samples in the count tab
# We can do this by reindexing the count table with the index of th
print(counts_df.index)
counts_df = counts_df.loc[meta.index]
print(counts_df.index)

```

```

Index(['Naive_EZH2i_R1', 'Naive_EZH2i_R2', 'Naive_EZH2i_R3', 'Naive_
      'Naive_R2', 'Naive_R3', 'Primed_EZH2i_R1', 'Primed_EZH2i_R2',
      'Primed_EZH2i_R3', 'Primed_R1', 'Primed_R2', 'Primed_R3'],
      dtype='object')
Index(['Naive_R1', 'Naive_R2', 'Naive_R3', 'Naive_EZH2i_R1', 'Naive_
      'Naive_EZH2i_R3', 'Primed_R1', 'Primed_R2', 'Primed_R3',
      'Primed_EZH2i_R1', 'Primed_EZH2i_R2', 'Primed_EZH2i_R3'],
      dtype='object', name='Name')

```

```

#####
##### TASK 3 #####
# Running DESeq2 #
#####

```

```

from pydeseq2.dds import DeseqDataSet
from pydeseq2.ds import DeseqStats

```

```

# create DeseqDataSet
dds = DeseqDataSet(
    counts=counts_df,
    metadata=meta,
    design=~design",
    refit_cooks=True)

```

```
# run dds.deseq2()
dds.deseq2()
```

```
Using None as control genes, passed at DeseqDataSet initialization
Fitting size factors...
... done in 0.02 seconds.

Fitting dispersions...
... done in 29.31 seconds.

Fitting dispersion trend curve...
... done in 1.16 seconds.

Fitting MAP dispersions...
... done in 28.86 seconds.

Fitting LFCs...
... done in 15.44 seconds.

Calculating cook's distance...
... done in 0.03 seconds.

Replacing 0 outlier genes.
```

```
#extract and log transform the normalized counts for PCA, it is sto
lnc = np.log1p(dds.layers["normed_counts"]) # samples x genes

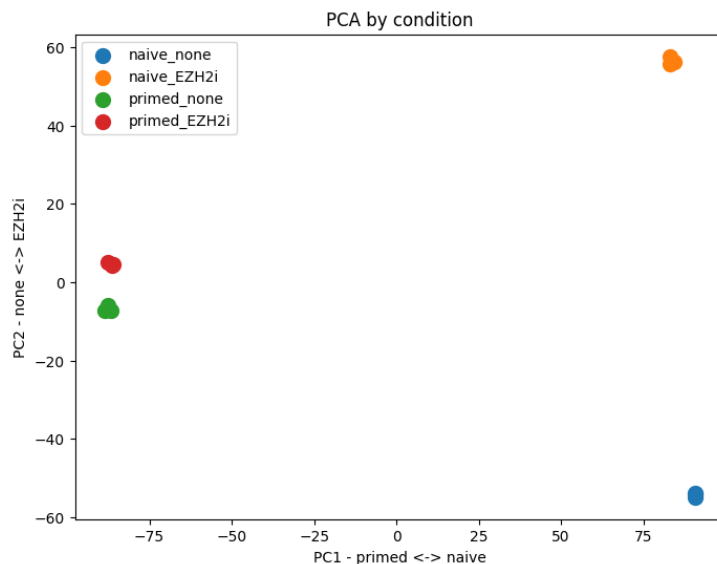
# Basic PCA with scikit-learn
from importlib.metadata import metadata

from sklearn.decomposition import PCA

X = np.asarray(lnc) # make sure it's a nume
pca = PCA(n_components=2).fit_transform(X) # PCs across samples

# Plot, color by design, provide legend and labels of points
plt.figure(figsize=(8, 6))
for design in meta["design"].unique():
    mask = meta["design"] == design
    plt.scatter(pca[mask, 0], pca[mask, 1], label=design, marker="o")

plt.xlabel("PC1 - primed <=> naive")
plt.ylabel("PC2 - none <=> EZH2i")
plt.title("PCA by condition")
plt.legend()
plt.show()
```



```
# now print the normalized read counts for GAPDH and NANOG from DES
dds.layers.keys() # check what layers are available in the dds obj
normed_counts = pd.DataFrame(dds.layers["normed_counts"], index=cou
print("GAPDH normalized read counts:", normed_counts.loc[:, "GAPDH"
print("NANOG normalized read counts:", normed_counts.loc[:, "NANOG"
```

```
GAPDH normalized read counts: Name
Naive_R1      22044.877568
Naive_R2      21297.458876
Naive_R3      22853.262509
Naive_EZH2i_R1 26170.432342
Naive_EZH2i_R2 27194.972841
Naive_EZH2i_R3 25210.889334
Primed_R1     44466.179739
Primed_R2     44787.843098
Primed_R3     49500.559470
Primed_EZH2i_R1 43075.690799
Primed_EZH2i_R2 46267.177219
Primed_EZH2i_R3 41171.720991
Name: GAPDH, dtype: float64
NANOG normalized read counts: Name
Naive_R1      1627.839289
Naive_R2      1660.494465
Naive_R3      1707.305618
Naive_EZH2i_R1 309.584763
Naive_EZH2i_R2 244.757740
Naive_EZH2i_R3 233.625837
Primed_R1     485.174247
Primed_R2     451.186058
Primed_R3     466.484206
Primed_EZH2i_R1 592.812970
Primed_EZH2i_R2 523.080569
Primed_EZH2i_R3 565.941201
Name: NANOG, dtype: float64
```

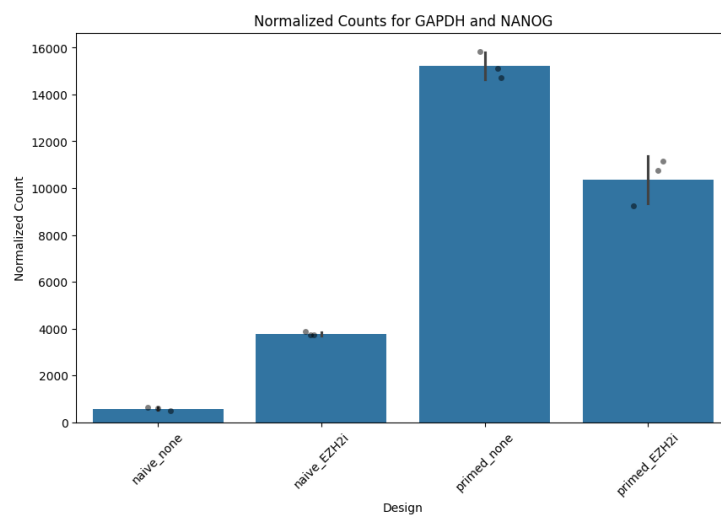
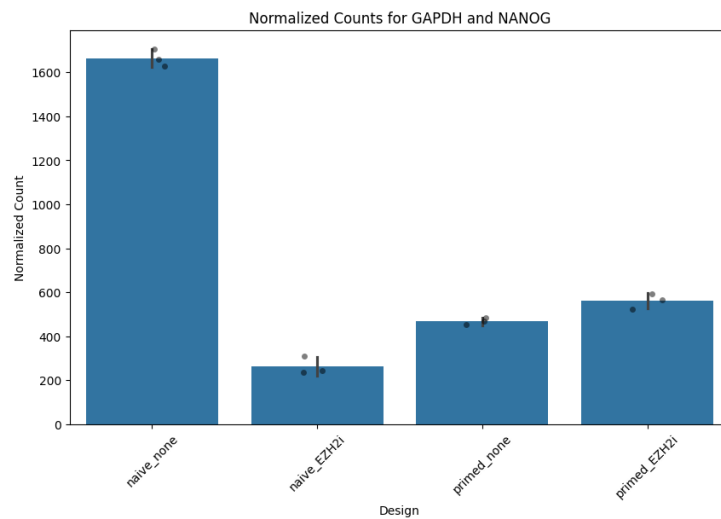
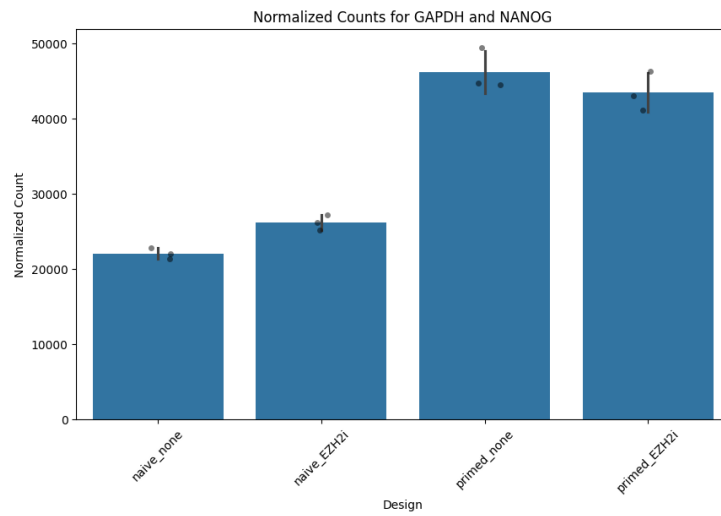
```
# let's make a nicer bar plot of the expressin values per design fo
# we need to annotate the rows to group them by design, we can do t
# melt, keep the Name and gene_identifier as columns, and the gene
normed_counts_long = normed_counts.reset_index().melt(id_vars=["Nam
normed_counts_long["design"] = normed_counts_long["Name"].map(meta[
print(normed_counts_long.head())
```

	Name	gene_id	normalized_count	design
0	Naive_R1	A1BG	4.184677	naive_none
1	Naive_R2	A1BG	0.000000	naive_none
2	Naive_R3	A1BG	8.494058	naive_none
3	Naive_EZH2i_R1	A1BG	9.017032	naive_EZH2i
4	Naive_EZH2i_R2	A1BG	11.939402	naive_EZH2i

```
# Plot normalized counts for GAPDH, use bar for mean and error bar
plt.figure(figsize=(10, 6))
sns.barpplot(data=normed_counts_long[normed_counts_long["gene_id"].i
sns.stripplot(data=normed_counts_long[normed_counts_long["gene_id"]
plt.xlabel("Design")
plt.ylabel("Normalized Count")
plt.title("Normalized Counts for GAPDH and NANOG")
plt.xticks(rotation=45)
plt.show()
```

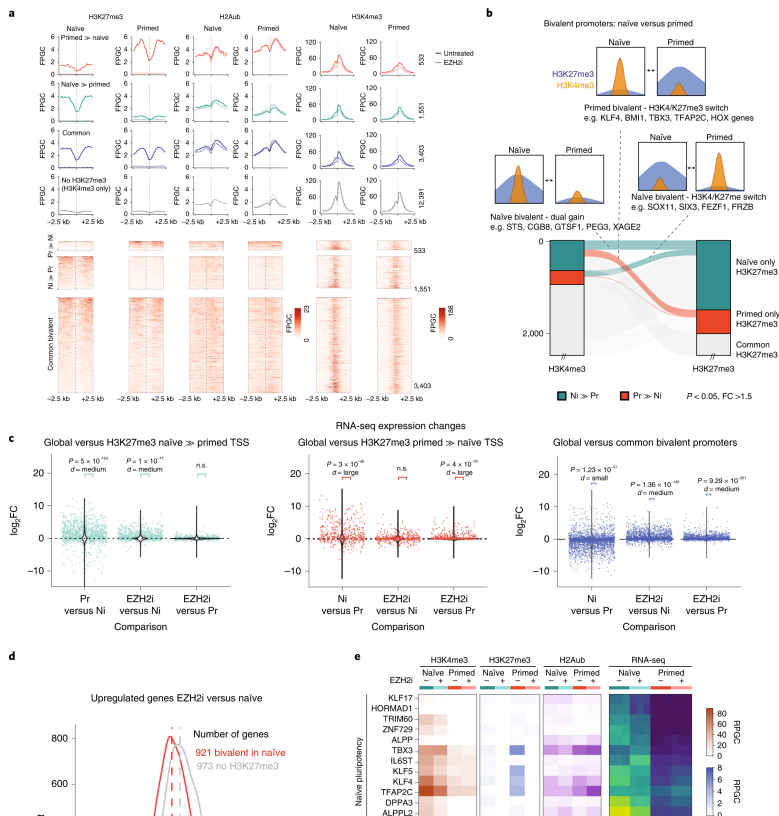
```
# Plot normalized counts for NANOG, use bar for mean and error bar
plt.figure(figsize=(10, 6))
sns.barpplot(data=normed_counts_long[normed_counts_long["gene_id"].i
sns.stripplot(data=normed_counts_long[normed_counts_long["gene_id"]
plt.xlabel("Design")
plt.ylabel("Normalized Count")
plt.title("Normalized Counts for GAPDH and NANOG")
plt.xticks(rotation=45)
plt.show()
```

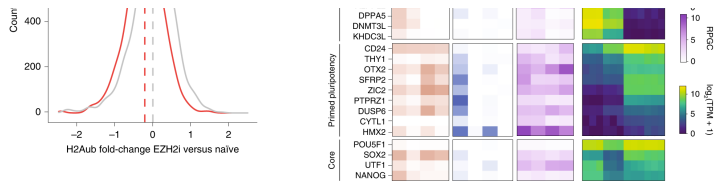
```
# Plot normalized counts for CD24, use bar for mean and error bar f
plt.figure(figsize=(10, 6))
sns.barplot(data=normed_counts_long[normed_counts_long["gene_id"]].i
sns.stripplot(data=normed_counts_long[normed_counts_long["gene_id"]
plt.xlabel("Design")
plt.ylabel("Normalized Count")
plt.title("Normalized Counts for GAPDH and NANOG")
plt.xticks(rotation=45)
plt.show()
```



```
#####
##### TASK 4 #####
# Heatmap Figure 2e #
#####
```

```
# Check out Figure 2e, the figure shows a selection of differential
# using log TPM values
```





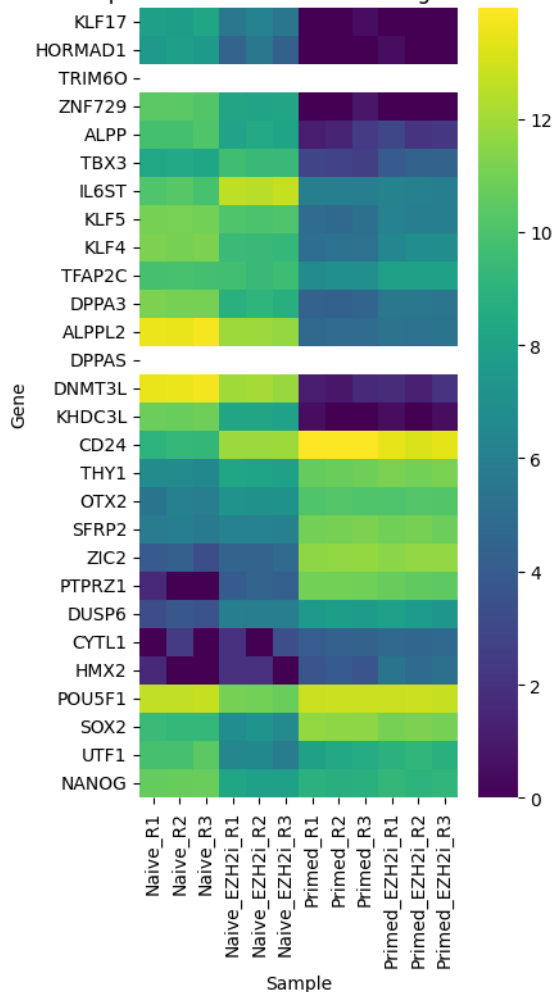
```
#####
##### TASK 4 #####
# Heatmap Figure 2e #
#####
```

```
genes_2e=["KLF17","HORMAD1","TRIM60","ZNF729","ALPP","TBX3","IL6ST"

#from normed_counts_long, subset for the genes in genes_2e and plot

normed_counts_log = normed_counts.reset_index().melt(id_vars=["Name
normed_counts_log["normalized_count"] = np.log2(normed_counts_log["
normed_counts_log["design"] = normed_counts_log["Name"].map(meta["d
normed_counts_2e = normed_counts_log[normed_counts_log["gene_id"]].i
normed_counts_2e_pivot = normed_counts_2e.pivot(index="gene_id", co
# reindex to maintain the original order from genes_2e
normed_counts_2e_pivot = normed_counts_2e_pivot.reindex(genes_2e)
# sort columns by design order: naive_none, naive_EZH2i, primed_non
design_order = ["naive_none","naive_EZH2i","primed_none","primed_EZ
sorted_columns = sorted(normed_counts_2e_pivot.columns,
                        key=lambda x: (design_order.index(meta.loc[x
normed_counts_2e_pivot = normed_counts_2e_pivot[sorted_columns]
# plot a heatmap of the normalized counts for these genes across th
plt.figure(figsize=(4, 8))
sns.heatmap(normed_counts_2e_pivot, cmap="viridis", yticklabels=True
plt.title("Heatmap of Normalized Counts for Figure 2e Genes")
plt.xlabel("Sample")
plt.ylabel("Gene")
plt.xticks(rotation=90)
plt.show()
```

Heatmap of Normalized Counts for Figure 2e Genes



```
#####
##### TASK 5 #####
```

```
#####
# volcano plots #
#####

# Extract the results of the DESeq2 analysis for the contrast "Naiv
# stored in the "results_df" attribute of the ds object

ds = DeseqStats(dds, contrast=["design", "naive_none", "primed_none
ds.summary()
naive_vs_primed_raw_lfc = ds.results_df["log2FoldChange"]
ds.lfc_shrink(coeff="design[T.naive_none]")

naive_vs_primed = ds.results_df
ds.summary()
```

gene_id					
A1BG	30.878081	-1.073793	0.720745	-5.638464	1.7151
A1BG-AS1	10.796228	-1.213623	0.712611	-1.732507	8.3183
A1CF	0.164028	-0.008273	0.601286	-0.007055	9.9437
A2M	26.315026	-0.714668	1.013293	-3.653862	2.5837
A2M-AS1	1.510204	-0.011719	0.578773	-1.551723	1.2077
...	...	...	...	...	...
ZYG11A	159.035116	2.083494	0.182893	27.046990	4.14394
ZYG11B	281.050682	-0.187746	0.129399	1.018290	3.0854
ZYX	2183.877283	0.018237	0.101830	3.953679	7.6958
ZZEF1	450.410347	0.269318	0.091534	18.474701	3.3005
ZZZ3	331.013195	0.254141	0.142510	0.990875	3.2174

	padj
gene_id	
A1BG	4.799909e-08
A1BG-AS1	1.092614e-01
A1CF	NaN
A2M	4.952732e-04
A2M-AS1	1.533186e-01
...	...
ZYG11A	3.397432e-159
ZYG11B	3.563518e-01
ZYX	1.561455e-04
ZZEF1	8.295009e-75
ZZZ3	3.700107e-01

```
[26475 rows x 6 columns]
Log2 fold change & Wald test p-value: design naive_none vs primed_r
      baseMean  log2FoldChange      lfcSE      stat
gene_id
A1BG          30.878081      -2.972687    0.720745    -5.638464    1.7151
A1BG-AS1      10.796228      -0.613118    0.712611    -1.732507    8.3183
A1CF           0.164028       0.449090    0.601286    -0.007055    9.9437
A2M           26.315026     -4.918365    1.013293    -3.653862    2.5837
A2M-AS1       1.510204     -2.773232    0.578773    -1.551723    1.2077
...
ZYG11A       159.035116      4.407479    0.182893    27.046990    4.14394
ZYG11B       281.050682      0.119662    0.129399     1.018290    3.0854
ZYX          2183.877283      0.395497    0.101830     3.953679    7.6958
ZZEF1        450.410347      1.434197    0.091534    18.474701    3.3005
ZZZ3         331.013195      0.113135    0.142510     0.990875    3.2174
```

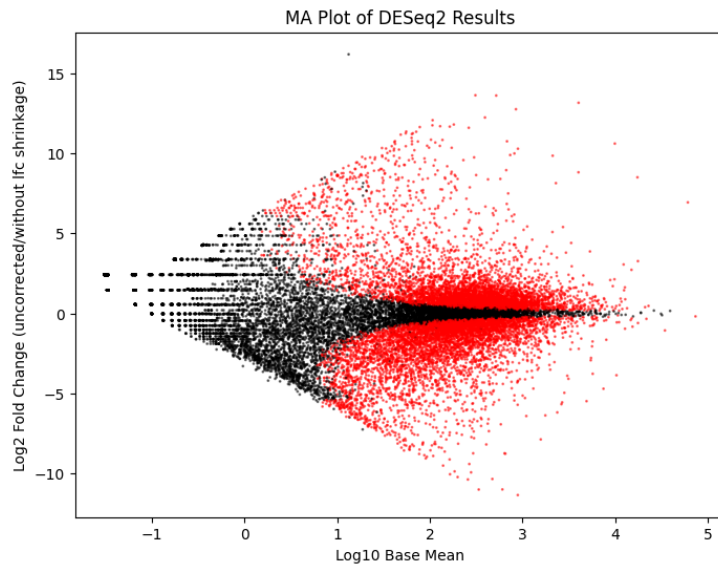
	padj
gene_id	
A1BG	4.799909e-08
A1BG-AS1	1.092614e-01
A1CF	NaN
A2M	4.952732e-04
A2M-AS1	1.533186e-01
...	...
ZYG11A	3.397432e-159
ZYG11B	3.563518e-01
ZYX	1.561455e-04
ZZEF1	8.295009e-75
ZZZ3	3.700107e-01

```
[26475 rows x 6 columns]
```

```
#####
# Plot a so-called MA-plot:
# xaxis is the expression of the gene (log10[baseMean])
# yaxis is the log2 fold change, color points with padj < 0.05 in r

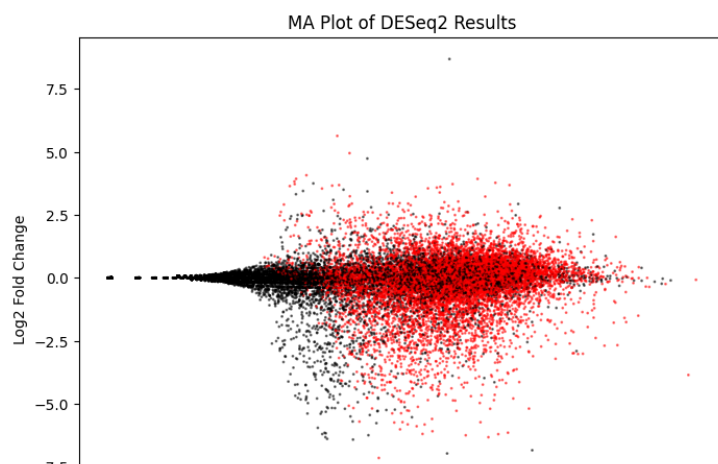
plt.figure(figsize=(8, 6))
colors = np.where(naive_vs_primed["padj"] < 0.01, "red", "black")
plt.scatter(np.log10(naive_vs_primed["baseMean"]), naive_vs_primed["log2FoldChange"], c=colors)
plt.xlabel("Log10 Base Mean")
plt.ylabel("Log2 Fold Change (uncorrected/without lfc shrinkage)")
plt.title("MA Plot of DESeq2 Results")
plt.show()
```

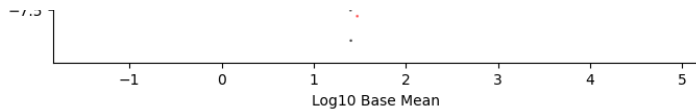
```
/usr/local/lib/python3.12/dist-packages/pandas/core/arraylike.py:399
result = getattr(ufunc, method)(*inputs, **kwargs)
```



```
#####
# looks like there are very high fold-changes for very lowly expres
# this is a common problem in DE analysis, because the fold change
# That's why we use a method called "lfcShrink" that shrinks the log
```

```
plt.figure(figsize=(8, 6))
colors = np.where(naive_vs_primed["padj"] < 0.01, "red", "black")
plt.scatter(np.log10(naive_vs_primed["baseMean"]), naive_vs_primed["log2FoldChange"], c=colors)
plt.xlabel("Log10 Base Mean")
plt.ylabel("Log2 Fold Change")
plt.title("MA Plot of DESeq2 Results")
plt.show()
```





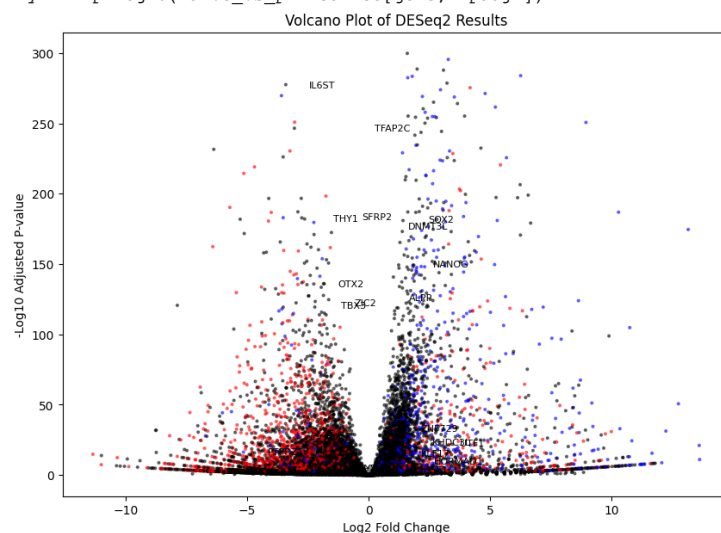
```
#####
# Volcano plot:
# xaxis is the log2 fold change, yaxis is the -log10 of the adjusted p-value

# print Volcano plot of the results, color points with padj < 0.01
plt.figure(figsize=(10, 7))

colors = np.where((naive_vs_primed["padj"] < 0.01) & (naive_vs_primed["log2FoldChange"] > 1), "red", "black")
plt.scatter(naive_vs_primed_raw_lfc, -np.log10(naive_vs_primed["padj"]), c=colors)
plt.xlabel("Log2 Fold Change")
plt.ylabel("-Log10 Adjusted P-value")
plt.title("Volcano Plot of DESeq2 Results")

# Add text labels for genes in genes_2e
texts = []
for gene in genes_2e:
    if gene in naive_vs_primed.index:
        x = naive_vs_primed.loc[gene, "log2FoldChange"]
        y = -np.log10(naive_vs_primed.loc[gene, "padj"])
        texts.append((x, y, gene, fontsize=8))
plt.show()
```

/tmp/ipykernel_9243/2728612721.py:18: RuntimeWarning: divide by zero
y = -np.log10(naive_vs_primed.loc[gene, "padj"])

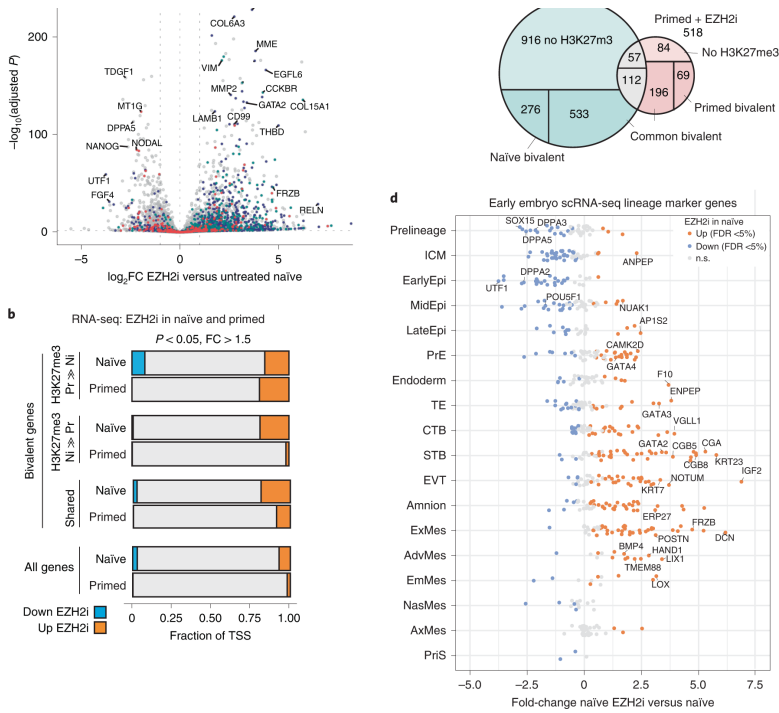


Remember:

Measuremen	Meaning
p-value	evidence against null
padj-valu	p after correction for multiple testing
log2FC	estimated effect size
shrunk log2FC	stabilized plausible effect size

Now lets look at Figure 2a! comparing EZH2i vs none in the "Nai





```
#####
##### TASK #####
# volcano plots #
#####
```

```
# Extract the results of the DESeq2 analysis for the contrast "Naiv
# stored in the "results_df" attribute of the ds object
```

```
ds = DESeqStats(dds, contrast=["design", "naive_EZH2i", "naive_none
ds.summary()
EZH2i_vs_naive_raw_lfc = ds.results_df["log2FoldChange"]
ds.lfc_shrink(coeff="design[T.naive_none])
```

```
EZH2i_vs_naive = ds.results_df
ds.summary()
```

gene_id	baseMean	log2FoldChange	lfcSE	stat	pvalue
A1BG	30.878081	-1.070979	0.722766	2.183363	2.90090e-05
A1BG-AS1	10.796228	-1.209334	0.714400	2.394446	1.66451e-05
A1CF	0.164028	-0.008115	0.594739	0.099575	9.20681e-05
A2M	26.315026	-0.697561	0.990770	1.381892	1.67001e-05
A2M-AS1	1.510204	-0.011466	0.572472	0.172228	8.63251e-05
...	...	...	...	...	...
ZYG11A	159.035116	2.083389	0.182909	-11.576604	5.41481e-29
ZYG11B	281.050682	-0.187612	0.129339	1.467490	1.42241e-01
ZYX	2183.877283	0.018227	0.101794	-0.181849	8.55701e-05
ZZEF1	450.410347	0.269230	0.091517	-2.955523	3.12131e-05
ZZZ3	331.013195	0.253932	0.142441	-1.804723	7.11181e-05

gene_id	padj
A1BG	6.274653e-02
A1BG-AS1	3.874012e-02
A1CF	NaN
A2M	2.673872e-01
A2M-AS1	NaN
...	...
ZYG11A	1.586067e-29
ZYG11B	2.362405e-01
ZYX	9.027050e-01
ZZEF1	8.817398e-03
ZZZ3	1.338606e-01

```
[26475 rows x 6 columns]
Log2 fold change & Wald test p-value: design naive_EZH2i vs naive_
baseMean log2FoldChange lfcSE stat pvalue
gene_id
A1BG 30.878081 1.070979 0.722766 2.183363 2.90090e-05
A1BG-AS1 10.796228 1.209334 0.714400 2.394446 1.66451e-05
A1CF 0.164028 0.008115 0.594739 0.099575 9.20681e-05
A2M 26.315026 0.697561 0.990770 1.381892 1.67001e-05
A2M-AS1 1.510204 0.011466 0.572472 0.172228 8.63251e-05
...
ZYG11A 159.035116 -2.083389 0.182909 -11.576604 5.41481e-29
ZYG11B 281.050682 0.187612 0.129339 1.467490 1.42241e-01
ZYX 2183.877283 -0.018227 0.101794 -0.181849 8.55701e-05
ZZEF1 450.410347 -0.269230 0.091517 -2.955523 3.12131e-05
ZZZ3 331.013195 -0.253932 0.142441 -1.804723 7.11181e-05
```

```

                padj
gene_id
A1BG      6.274653e-02
A1BG-AS1  3.874012e-02
A1CF      NaN
A2M       2.673872e-01
A2M-AS1   NaN
...
ZYG11A    1.586067e-29
ZYG11B    2.362405e-01
ZYX       9.027050e-01
ZZEF1     8.817398e-03
ZZZ3      1.338606e-01

[26475 rows x 6 columns]
done in 63.08 seconds

```

```

# genes to mark as in volcano plot
genes_3a = ["KRT18", "H19", "IGF2", "EPAS2", "COL6A3", "MME",
            "VIM", "MMP2", "EGFL6", "CCKBR", "GATA2", "COL15A1", "CD99", "EPAS2",
            "LAMB1", "THBD", "FRZB", "RELN", "TDGF1", "MT1G", "DDPA5", "NODAL",
            "FGF4"]

#####
# Volcano plot:
# xaxis is the log2 fold change, yaxis is the -log10 of the adjusted

# print Volcano plot of the results, color points with padj < 0.01
plt.figure(figsize=(10, 7))
colors = np.where((EZH2i_vs_naive["padj"] < 0.01) & (EZH2i_vs_naive["log2FoldChange"] > 1),
                  "blue", "red")
plt.scatter(EZH2i_vs_naive["log2FoldChange"], -np.log10(EZH2i_vs_naive["padj"]), c=colors)
plt.xlabel("Log2 Fold Change")
plt.ylabel("-Log10 Adjusted P-value")
plt.title("Volcano Plot of DESeq2 Results")

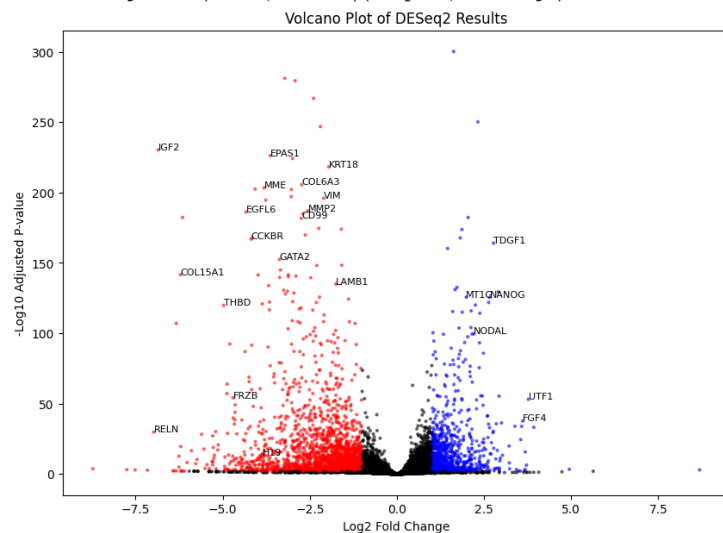
# Add text labels for genes in genes_2e
texts = []
for gene in genes_3a:
    if gene in EZH2i_vs_naive.index:
        x = EZH2i_vs_naive.loc[gene, "log2FoldChange"]
        y = -np.log10(EZH2i_vs_naive.loc[gene, "padj"])
        texts.append(plt.text(x, y, gene, fontsize=8))
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/pandas/core/arraylike.py:399
result = getattr(ufunc, method)(*inputs, **kwargs)

```



```

topEZH2i = EZH2i_vs_naive[(EZH2i_vs_naive["padj"] < 0.01) & (EZH2i_vs_naive["log2FoldChange"] > 1)]

# from normed_counts_long, subset for the genes in genes_2e and plot a
normed_counts_3a = normed_counts_log[normed_counts_log["gene_id"].isin(topEZH2i["gene_id"])]

```

```
normed_counts_3a_pivot = normed_counts_3a.pivot(index="gene_id", col
# sort columns by design order: naive_none, naive_EZH2i, primed_none,
design_order = ["naive_none", "naive_EZH2i", "primed_none", "primed_EZH2i"]
sorted_columns = sorted(normed_counts_3a_pivot.columns,
                        key=lambda x: (design_order.index(meta.loc[x,
normed_counts_3a_pivot = normed_counts_3a_pivot[sorted_columns])
# plot a heatmap of the normalized counts for these genes across the
plt.figure(figsize=(4, 8))
sns.heatmap(normed_counts_3a_pivot, cmap="viridis", yticklabels=True)
plt.title("Heatmap of Normalized Counts for Genes most significantly
plt.xlabel("Sample")
plt.ylabel("Gene")
plt.xticks(rotation=90)
plt.show()
```

